

Complete SDLC Lifecycle with Detailed Explanations

1. Introduction

Software Development Life Cycle (SDLC) is a structured framework used to plan, design, build, test, deploy, and maintain software systems. It improves quality, predictability, cost control, and customer satisfaction.

2. Planning

Planning identifies business goals, project scope, stakeholders, timelines, budget, risks, and resource requirements. Deliverables include the Project Charter, feasibility report, and high-level schedule.

3. Requirements Engineering

Business Analysts gather functional and non-functional requirements using interviews, workshops, observations, questionnaires, and document analysis. Outputs include BRD, SRS, user stories, use cases, and acceptance criteria.

4. System Design

Architects prepare High-Level Design (HLD) describing architecture, technology stack, modules, integrations, security, and deployment. Developers create Low-Level Design (LLD) including class diagrams, database schema, APIs, algorithms, and validations.

5. Development

Developers implement features following coding standards, version control, peer reviews, secure coding practices, and unit testing. CI tools automate builds and quality checks.

6. Testing

QA performs Unit, Integration, System, Regression, Performance, Security, and User Acceptance Testing. Defects are tracked, prioritized, fixed, and retested.

7. Deployment

Applications are released through staging and production environments using CI/CD pipelines. Activities include configuration management, database migration, rollback planning, and release validation.

8. Maintenance

After release, teams monitor performance, fix defects, optimize performance, enhance functionality, and address security vulnerabilities. Maintenance can be corrective, adaptive, perfective, or preventive.

9. Roles

Business Analyst, Product Owner, Solution Architect, Developers, QA Engineers, DevOps Engineers, Scrum Master, Product Manager, Project Manager, and Operations collaborate throughout the lifecycle.

10. Best Practices

Maintain traceability from requirements to testing, automate builds and deployments, perform code reviews, write comprehensive tests, document architecture, and continuously gather customer feedback.

11. Real-world Example

A banking application begins with business requirements, designs secure APIs and database models, develops account and transaction modules, performs rigorous security testing, deploys through CI/CD, and continuously enhances features based on customer feedback.

12. Interview Questions

Explain SDLC phases; differentiate HLD and LLD; compare Waterfall and Agile; explain verification vs validation; discuss CI/CD benefits; describe requirement traceability; explain maintenance types.